

Vision Toolkit Architecture

System design · Data flow · Component breakdown

Internship
AI Summer 2026

Track
Computer Vision

Week
1 of 8

Version
1.0

1. System Architecture Overview

The Vision Toolkit architecture is structured as an event-driven, stateless-per-call data pipeline. The workflow begins with the browser-based User Interface, passes through sequential image decoding, executes modular OpenCV filters directly on original matrices, and displays synchronized side-by-side visualizations before generating an in-memory downloadable asset.

2. Stage-by-Stage Explanation

Stage 01: User

Entry point · Browser interface

The user interacts with Vision Toolkit entirely through a Streamlit web interface running in any modern browser. No installation is required beyond Python. The user's actions drive the entire pipeline: uploading an image triggers data loading; selecting a sidebar operation triggers a processing call; clicking download triggers export. All state is managed in `st.session_state`, persisting the image between sidebar interactions.

Stage 02: Image Input

Acquisition · Decoding · Metadata

When a file is uploaded, Streamlit's File API returns raw bytes. These bytes are decoded with `cv2.imdecode()` into a NumPy array of shape (H, W, 3) — the core data structure OpenCV works with. File metadata (name, size, format, dimensions, channels) is extracted and displayed in the info bar. Two copies of the array are stored: the untouched original and the working copy.

Stage 03: OpenCV Processing Engine

Six operation modules · Stateless per call · Always reads original

This is the core of the application. Six processing modules handle all image operations. A critical design choice: every operation reads from the original image, not from the previously processed result. This

prevents quality loss from chained operations and lets any filter be previewed or changed instantly without permanent side effects.

- Color module — converts between color spaces using `cv2.cvtColor()`. Brightness and contrast adjust pixel values via `convertScaleAbs(alpha, beta)`.
- Blur module — slides a kernel across the image using convolution. Gaussian uses a bell-curve kernel; Median replaces each pixel with the sorted middle value (better for noise). Kernel size must always be odd — enforced by `ensure_odd()`.
- Threshold module — converts to grayscale first, then binarizes. Adaptive threshold computes a local threshold per pixel block, making it robust to uneven lighting.
- Edge module — runs Canny's 5-step pipeline: blur → Sobel gradients → gradient magnitude → non-maximum suppression → hysteresis.
- Transform module — rotation uses `getRotationMatrix2D()` followed by `warpAffine()`. Resize uses bilinear interpolation.
- Drawing module — annotates the image copy using `cv2.rectangle()`, `cv2.circle()`, `cv2.line()`, `cv2.putText()`. Hex color from the color picker is converted to BGR via a helper function.

Stage 04: Visualization

Display · Histogram · Metrics

The processed NumPy array is converted from BGR → RGB before display (OpenCV stores channels in reverse order; Streamlit/matplotlib expect RGB). Original and processed images are shown side by side for instant comparison. The histogram panel uses `cv2.calcHist()` to compute 256-bin intensity distributions for each channel, then renders them with matplotlib, embedded via `st.pyplot()`. Image metrics (width, height, channels, file size) are shown in metric cards.

Stage 05: Output

Export · PNG encoding · Download

The processed NumPy array is converted to a PIL Image object, then encoded into PNG bytes using an in-memory `io.BytesIO` buffer — no temporary file is written to disk. Streamlit's `st.download_button()` presents these bytes as a downloadable file. The user receives a lossless PNG preserving all pixel-level changes made during processing.

3. Data Flow — Input & Output per Stage

Stage	Input	Output	Data Format
01 User	Selects image file, chooses operation	File object + UI events	UploadedFile · widget values
02 Image Input	Raw file bytes from browser	Original image array stored in	np.ndarray (H, W, 3) uint8

		session	BGR
03 Processing	Original array + operation parameters	Processed image array	np.ndarray (H', W', C') uint8
04 Visualization	Processed array + original array	Rendered images + histogram figures	RGB numpy · matplotlib Figure
05 Output	Processed array	PNG file download to user's device	io.BytesIO → image/png bytes

4. Technology Stack

- **Python 3.11:** Core language · Application logic · Data flow
- **OpenCV 4.9:** All image processing · Color · Blur · Edge · Draw
- **Streamlit:** Web UI · Widgets · State management · File I/O
- **NumPy:** Image arrays · Pixel math · Array operations
- **Matplotlib:** Histogram rendering · Channel plots
- **Pillow:** Image encoding · PNG byte export
- **GitHub:** Version control · Code hosting · Submission

5. Key Design Decisions

Why always read from the original image?

Each operation reads the unmodified original, not the previously processed result. This prevents quality degradation from chained operations (e.g., blurring a blurred image loses more detail each time) and means any operation can be changed or previewed without side effects.

Why convert BGR → RGB before display?

OpenCV loads images in Blue-Green-Red channel order for historical reasons. Streamlit and matplotlib expect Red-Green-Blue. Without conversion, red and blue channels are swapped — a photo of a red apple would appear blue. `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` corrects this at display time only.